

정규화와 모델 선택: Lasso/Ridge · 교차검증 · train/val/test

6.1 정규화와 L1/L2 페널티 · 6.2 교차검증으로 λ 를 데이터로 정한다 · 6.3 3분할 원칙 실전

Machine Learning 1 · 6주차

한국과학영재학교 (KSA)

Chapter 6

이번 주차 개요

1996년 티브시라니(Tibshirani)의 한 생각 — 손실함수에 가중치 절댓값의 합을 페널티로 더하면, 무관한 특징의 가중치가 **정확히 0**이 된다. 특징이 수천 개여도 사람이 일일이 고를 필요가 없다.

이 챕터를 마치면

- “가중치가 작다 = 모델이 단순하다 = 분산이 작다”의 연결고리를 설명하고, L2는 매끄러운 축소(shrinkage)를, L1은 **정확히 0**으로 특징 선택까지 해내는 이유를 짚음
- λ 를 편향-분산 트레이드오프 위의 **손잡이**로 해석하고, 계수 경로(coefficient path) 실험으로 확인
- k-겹 교차검증으로 λ 후보를 검증 MSE로 비교하고, U자형 곡선의 좌측 = 과적합 · 우측 = 과소적합으로 읽음
- train/val/test 3분할 원칙을 적용하고, “정보 흐름의 역주행” (leakage)이 있는 코드를 진단

세 차시 한 줄

- **6.1** 페널티의 “모양”이 결과를 바꾼다 — 원(L2) 대 마름모(L1), soft-thresholding
- **6.2** 학습 손실이 아니라 **검증** 손실로 λ 를 고른다: k-fold CV · 1SE 규칙 · 최적화 편향
- **6.3** train은 배운다, val은 고르고, test는 **딱 한 번** — 정보 흐름에 방벽을 세운다

이미 써본 그 손잡이: Ch. 5 소프트 마진 SVM의 목적함수 $\frac{1}{2}\|w\|^2 + C \sum_i \xi^{(i)}$ 에서 $\|w\|^2$ 자체가 L2 페널티, C 는 이 장의 λ 와 같은 종류의 손잡이었다.

6.1 정규화: 편향-분산 복습과 L1/L2 페널티

손실함수에 페널티 항을 더한다

Ch. 4.1에서 봤듯 모델이 너무 유연하면(파라미터가 크고 자유로우면) 분산이 커져 **과적합**한다.
정규화(regularization)는 손실함수에 “가중치가 너무 커지지 않도록” 페널티 항을 더해 유효 유연성을 인위적으로 낮추는 기법이다:

$$J(w) = \underbrace{\frac{1}{2m} \sum_{i=1}^m (h_w(x^{(i)}) - y^{(i)})^2}_{\text{원래 손실(적합도)}} + \underbrace{\lambda R(w)}_{\text{정규화 항(단순함을 요구)}}$$

- λ 가 0이면 원래 손실과 같고(정규화 없음), 커질수록 “데이터에 딱 맞추기”보다 “가중치를 작게”를 더 중요시.
- 4.1의 언어로: λ 를 올리면 **분산은 줄고 편향은 늘어난다**.

이 절의 순서: (1) “가중치 작음 = 단순함”이 왜 성립, (2) L2·L1이 구체적으로 무슨 일, (3) L1이 정확히 0을 만드는 대수·기하 이유, (4) λ 가 편향-분산 위의 “손잡이”인지 숫자로 확인.

왜 “가중치가 작다”가 “모델이 단순하다”인가

핵심 메커니즘

특징을 평균 0·표준편차 1로 표준화해 두면, x_j 가 Δ 만큼 변할 때 예측치는 $|w_j| \Delta$ 이상으로 변할 수 있다 — $|w_j|$ 는 “특징 x_j 가 출력을 흔들어놓을 수 있는 **강도**”다.

모든 $|w_j|$ 가 작으면 입력이 아무리 요동쳐도 출력은 거의 못 흔들린다. 4.1에서 “노이즈 한 점에 예측이 크게 출렁이는 모델” = 분산이 큰 모델이었는데, 가중치가 작은 모델은 그 **정 반대**(흔들림이 억눌림) → 분산이 작다.

- “가중치 축소(shrinkage) = 유효 복잡도 감소”가 정규화의 핵심.
- 이 관점 때문에 가중치 크기 페널티는 deep learning에서 **weight decay**(가중치 감쇠)로 그대로 사용된다.

역사: Ridge는 1970년 마퀴트(불량 최소제곱에 이미 1962년부터 사용)와 호엘·케나드(λI 를 더한 행렬이 손실면에 “등정(ridge) 모양” 돌출)의 이름. Lasso는 26년 뒤, 1996년 티브시라니.

주의: 이 연결은 특징 스케일이 같을 때만 성립 → Ridge/Lasso 전에 스케일링을 먼저 하는 것이 실전 표준.

페널티의 모양이 다르면 결과도 다르다

가장 흔한 두 페널티:

L2 정규화 (Ridge)

$$R(w) = \|w\|_2^2 = \sum_j w_j^2$$

모든 가중치를 **매끄럽게 0** 쪽으로 축소(shrinkage). 좀처럼 정확히 0이 되지는 않는다.

둘 다 “가중치를 작게 만든다”는 목표는 같지만, **모양이 다르면 결과도 다르다.**

L1 정규화 (Lasso)

$$R(w) = \|w\|_1 = \sum_j |w_j|$$

중요하지 않은 가중치를 **정확히 0**으로 만들어버리는 경향. 그래서 정규화 + **특징 선택**을 자동 수행.

L1은 왜 0으로 “뭉개지는”가 — soft-thresholding

- L2의 미분 λw_j 는 현재 가중치에 비례 — 0이 아니면 미분은 0이 아님. 0에 가깝게만 만들 뿐, 0에서 **붙잡아두는** 힘은 없다.
- L1의 $|w_j|$ 는 0에서 뽕족 → 미분불가능. 0에서의 **서브그래디언트**는 구간 $[-1, 1]$, “0이 최적점”의 필요조건:

$$|\text{원래 손실의 기울기}(w_j=0 \text{ 에서})| \leq \lambda$$

- 원래 손실이 w_j 를 0에서 밖으로 당기려는 힘이 λ 보다 약하면 w_j 는 **0에 박혀 나오지 않는다.**

soft-thresholding 공식과 좌표하강

$$w_j \leftarrow \mathcal{T}_\lambda(z_j) = \text{sign}(z_j) \max(|z_j| - \lambda, 0)$$

z_j 는 “페널티 없는 세계에서 w_j 에 걸린 당기힘”(나머지 고정, w_j 만 최적 적합). 행동 세 줄:

조건	결과
$ z \leq \lambda$	0 (0으로 스냅)
$z > \lambda$	$z - \lambda$ (λ 만큼 축소)
$z < -\lambda$	$z + \lambda$

soft-thresholding: “신호가 약하면 0으로 스냅, 강하면 초과분만 통과”.

왜 좌표하강(coordinate descent)

$|w|$ 가 미분불가능해 **구배하강을 그대로 쓸 수 없으므로**, 한 번에 w_j 하나만 이 공식으로 갱신하고 나머지는 고정. scikit-learn Lasso가 내부적으로 하는 일이 이 공식의 반복.

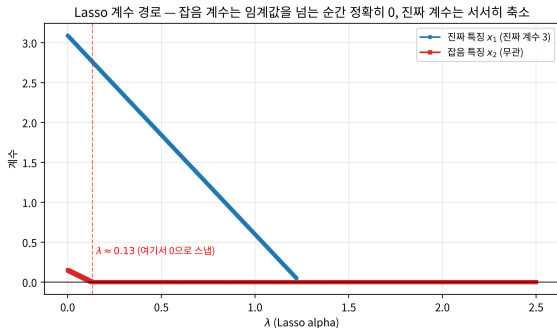
계수 경로: 0은 점진적이지 않고 “도착지”

“Lasso는 무관한 계수를 정확히 0으로 만든다”를 경로 전체로: λ 를 0에서 서서히 키우며 계수를 기록. 데이터: 30개 점, $x_1 \sim U(-1, 1)$, $y = 3x_1 + \text{잡음}(\sigma=1)$, $x_2 \sim N(0, 1)$ (y 와 무관한 잡음).

$\lambda = 0$ (OLS)에서 $x_1 \approx 3.09$, $x_2 \approx 0.15$.

진짜 x_1 : 3.09에서 점점 줄어, 잡음 스냅
(≈ 0.13)보다 훨씬 늦은 $\lambda \approx 1.2$ 에서야 정확히
0.

잡음 x_2 : 0.15에서 시작해 $\lambda \approx 0.13$ 넘는 순간
정확히 0, 그 뒤 한 번도 0에서 벗어나지 않음.



기하학적 직관: 원 vs 마름모

제약 $R(w) \leq t$ 에서 원래 손실을 최소화하는 문제로 다시 쓰면:

- **L2**: 제약 영역은 **원**(구) — 등고선이 표면 어디든 만남.
- **L1**: 제약 영역은 **마름모**(각 축에 뾰족한 꼭짓점) — 등고선이 **꼭짓점**(어떤 좌표가 0인 지점)에서 만날 확률이 훨씬 높음.

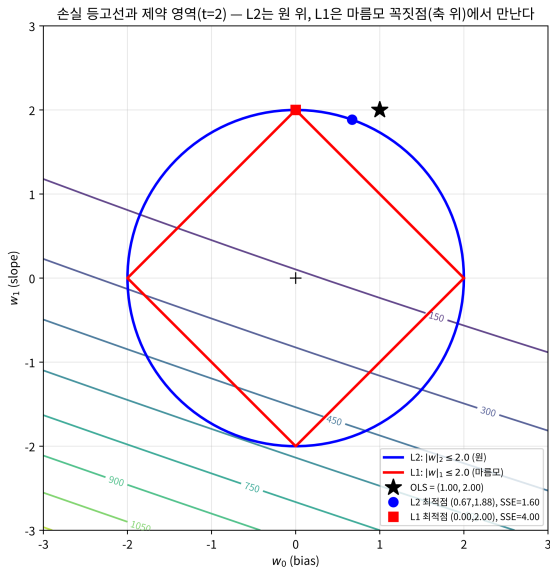
2차원 숫자: $X = [1, 2, 3, 4]$, $y = [3, 5, 7, 9]$ (직선 $y = 2x + 1$, OLS 해 $(1, 2)$)에서 제약 $t = 2$:

	최적점	SSE
L2(원)	$(0.67, 1.88)$ — 둘 다 0 아님	≈ 1.60
L1(마름모)	$(0, 2.0)$ — w_0 축 꼭짓점	≈ 4.00

차원이 클수록 희소해진다

2차원에선 꼭짓점이 4개뿐이지만 특징 p 개면 p 차원 정다면체로 변해 꼭짓점(=어떤 좌표가 0인 점)이 **방대해짐**. 특징이 많을수록 “등고선이 꼭짓점에서 만남” 확률이 커지므로, Lasso의 희소(sparse)한 해는 **차원이 클수록 더 흔해진다**.

L1 최적점은 꼭짓점에 닿는다



Ridge 경사하강 구현

```
def ridge_gradient_descent(X, y, lam, alpha, epochs):  
    m, n = len(X), len(X[0])  
    w = [0.0] * (n + 1) # w[0] = bias  
    for _ in range(epochs):  
        grad = [0.0] * (n + 1)  
        for i in range(m):  
            pred = w[0] + sum(w[j+1] * X[i][j] for j in range(n))  
            error = pred - y[i]  
            grad[0] += error  
            for j in range(n):  
                grad[j+1] += error * X[i][j]  
        for j in range(n + 1):  
            # bias(w[0])는 관례적으로 정규화에서 제외  
            reg = lam * w[j] if j > 0 else 0  
            w[j] -= alpha * (grad[j] / m + reg)  
    return w
```

λ 를 0에서 키우면 학습된 가중치가 0에 가까워짐 — 같은 데이터에 $\lambda = 0, 0.5, 5.0$ 을 넣으면 기울기 w_1 이 $2.0 \rightarrow 1.4 \rightarrow 0.4$.

손으로 한 번: Ridge 정규방정식

Ch. 2.2의 정규방정식은 정규화 항이 있어도 닫힌 형태로 확장:

$$w^* = (X^T X + \lambda D)^{-1} X^T y \quad (D \text{는 bias 제외 대각행렬})$$

$X = [1, 2, 3], y = [3, 5, 7]$ (정규화 없는 해 $w^* = [1, 2]$)에 $\lambda = 2$:

$$X^T X + \lambda D = \begin{bmatrix} 3 & 6 \\ 6 & 14 \end{bmatrix} + \begin{bmatrix} 0 & 0 \\ 0 & 2 \end{bmatrix} = \begin{bmatrix} 3 & 6 \\ 6 & 16 \end{bmatrix}$$

행렬식 $3 \times 16 - 6 \times 6 = 12$, $X^T y = [15, 34]$ 에 곱하면 $w^* = [3, 1]$.

- 기울기 w_1 : $2 \rightarrow 1$ (“작게” 만들려는 효과).
- bias w_0 : $1 \rightarrow 3$ 으로 **커짐** — 기울기가 늘린 만큼 절편이 데이터에 맞추며 보상.
- $\lambda = 10$ 이면 $w^* \approx [4.33, 0.33]$ 으로 기울기가 더 0에 가까워짐.

자주 하는 실수: bias(절편)까지 정규화

코드가 $\text{reg} = \text{lam} * w[j] \text{ if } j > 0 \text{ else } 0$ 로 bias를 굳이 건너뛰는 이유. bias까지 정규화($D = I$)하면 같은 $\lambda = 2$ 에서 $w^*_{\text{wrong}} = (X^T X + 2I)^{-1} X^T y \approx [0.818, 1.818]$ — bias가 1에서 0.818로 **불필요하게** 끌려 내려감.

λ	올바른 $D = \text{diag}(0, 1)$	틀린 $D = I$ (bias도 정규화)
2	[3.00, 1.00]	[0.82, 1.82]
10	[4.33, 0.33]	[0.57, 1.28]

왜 안 좋은가

bias는 “모델 복잡함”이 아니라 데이터 전체의 **평균적 높이**. 기울기가 늘리면 bias가 보상하며 더 커져야 한다(3.00→4.33)는데, bias까지 정규화하면 λ 가 커질수록 0 쪽으로 반대 방향으로 끌려가(0.82→0.57) y 의 평균이 원점에서 먼 데이터(주택 가격)에서 체계적으로 아래로 치우친다.

scikit-learn Ridge, Lasso 등은 기본적으로 bias/intercept를 정규화 대상에서 제외.

유명한 사례: Lasso는 유전체학에서 태어났다

1996년 티브시라니 논문에서 이 방법을 검증한 데이터가 바로 **유전자 발현(genomics) 데이터**.

- 수백 개의 후보 유전자 중 실제 발현에 관여하는 소수를 “수동으로” 고르는 것은 불가능에 가까움.
- 특징 수(수백)가 샘플 수에 버금가는 문제에서, Lasso가 몇 개의 유전자만을 **정확히 0이 아닌** 계수로 남김.
- 이 “특징이 많고 진짜 신호는 소수” 문제 구조는 30년 뒤에도 그대로 — 전장 유전체 연관 분석(수백만 SNP에서 원인 변이), 희소 신호 추정, 텍스트(수십만 단어 중 일부만 관련).

Lasso 계열이 1차 후보로 쓰이는 이유: soft-thresholding이 약한 계수를 0으로 스냅하는 성질 그 자체.

실습: L1과 L2의 “정확히 0” 차이

```
from sklearn.linear_model import Ridge, Lasso
x1 = np.array([1,2,3,4,5,6], dtype=float)
x2 = np.array([5,3,8,1,9,2], dtype=float) # 와y 무관한잡음
y = 2 * x1 + 1
X = np.column_stack([x1, x2])
for alpha in [0.1, 1.0, 5.0]:
    ridge = Ridge(alpha=alpha).fit(X, y)
    lasso = Lasso(alpha=alpha).fit(X, y)
    print(f"alpha={alpha}: ridge x2={ridge.coef_[1]:.4f}, "
          f"lasso x2={lasso.coef_[1]:.4f}")
```

Ridge: x_2 계수는 점점 작아지지만 **정확히 0이 아님** **Lasso:** 세 α 모두 x_2 계수를 **정확히 0.0000**로 —
(-0.0004 $\rightarrow$ -0.0040 $\rightarrow$ -0.0153) 잡음 특징을 아예 “제거”
실전 데이터에서 Lasso가 이런 특징들의 계수를 0으로 만들어주는 것이 곧 “특징 선택을 자동으로 해낸다”의 실제 의미.

한 뼉 더: 등가 특징 두 개면 어느 쪽을 남기는가

x_1 과 완전히 같은 정보를 담는 등가 특징 $x_{2c} = 3x_1 - 4$ (완전 상관)를 추가 + 잡음 x_3 :

```
x2c = 3 * x1 - 4.0          # 과x1 등가완전( 상관) 특징
x3  = np.array([2,9,1,7,3,6], dtype=float)
Xc  = np.column_stack([x1, x2c, x3])
print(Lasso(alpha=2.0).fit(Xc, y).coef_)
# [0. 0.5905 0.] 순서 (: [x1, x2c, x3])
```

놀라운 결과: Lasso는 진짜 계수 2를 가진 x_1 을 0으로 만들고, 등가인 x_{2c} 를 남김(계수 0.59, 환산 기울기 ≈ 1.77).

“어느 쪽을 남길지”는 데이터가 결정하지 않는다 — 등가이면 손실함수가 그 방향으로 평탄해져 해가 유일하지 않고, 솔버가 어느 꼭짓점에 먼저 닿느냐에 따라 둘 중 하나가 임의로 낙점.

대조: Ridge는 상관 특징들의 가중치를 나눠 갖고, Lasso는 하나를 통째로 삭제 — 안정적 해 vs 희소 해, 둘 다 “합법적”.

λ 는 편향-분산 트레이드오프 위의 “손잡이”

Ch. 4.1의 분해(다시 인용). kNN의 유연도 손잡이가 k 였다면 선형모델의 손잡이는 λ :

$$\mathbb{E} \left[(y - \hat{f}(x))^2 \right] = \underbrace{\left(\text{Bias}[\hat{f}(x)] \right)^2}_{\text{구조적으로 못 맞추는 정도}} + \underbrace{\text{Var}[\hat{f}(x)]}_{\text{훈련이 바뀔 때 예측이 흔들림}} + \underbrace{\sigma^2}_{\text{데이터의 잡음}}$$

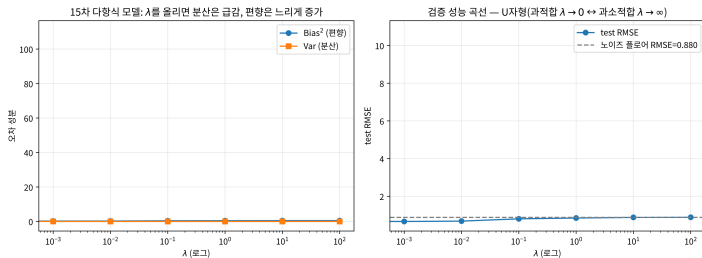
4.1의 수치 실험($f^*(x) = \sin(2\pi x)$, 잡음 $\sigma = 0.5$, 100점, 300회 재추출)과 **완전히 같은 방식**으로 재측정. 이번엔 모델이 **15차 다항식**(충분히 유연, $\lambda = 0$ 이면 100점을 거의 통과), “유연도 줄이는 조작”이 λ 를 올리는 것.

숫자로 확인: λ 축을 훑는다

λ	Bias ²	Var	test RMSE
0	5.64	110.90	10.81
0.001	0.24	0.030	0.66
0.01	0.26	0.015	0.68
0.1	0.40	0.008	0.80
1.0	0.44	0.004	0.85
10	0.48	0.004	0.88
100	0.49	0.004	0.89

- $\lambda = 0$: “주사위”처럼 출렁. Var = 110.9로 극도로 크고 RMSE 10.81은 **잡음 플로어(0.88)의 12배**. 과적합이 λ 축의 왼쪽 끝.
- $\lambda = 0.001$: Var 110.9 → 0.03으로 4자리수 급감, Bias² 0.24로 아직 작음. RMSE 0.66 = **곡선의 바닥**. “분산 제거”에 **극도로 효율적**.
- $\lambda = 100$: 가중치 완전히 눌러 “훈련 평균값만 내뱉는” 수준. Bias² 0.49, RMSE 0.89로 잡음 플로어에 붙음 — “ $\lambda \rightarrow \infty$ 이면 평균값만 예측하는 모델”의 숫자 확인.

U자형 곡선과 잡음 플로어



좌측: λ (로그 축)에 따른 Bias²와 Var — Var는 λ 를 올리자 급감, Bias²는 느리게 상승. 우측: test RMSE 곡선(U자형)과 잡음 플로어(0.88, 회색 점선).

U자형 곡선

왼쪽 날개 = 과적합(Var 지배), 오른쪽 날개 = 과소적합(Bias² 지배), 바닥 = 최적 λ 의 대략적 위치.

이 U자의 바닥을 “정직하게” 찾는 방법 = 6.2(교차검증)의 존재 이유. 검증 데이터가 없으면 “아마 이 부근”까지만 알 수 있다.

자주 묻는 질문 (1)

Q. 항상 Lasso(L1)를 쓰는 게 낫나요?

- 아님. 특징이 강하게 상관되면(집 크기 대 방 개수) Lasso는 하나만 임의로 남기고 나머지를 0으로 — **불안정**(데이터 살짝 바뀌도 남는 특징이 달라질 수 있음).
- Ridge는 상관 특징들의 가중치를 비슷하게 나눠 갖는 경향 → **안정적**.
- 실전: 둘을 섞은 **엘라스틱넷**(Zou·Hastie 2005), $R(w) = \alpha\|w\|_1 + (1 - \alpha)\|w\|_2^2$.

Q. Lasso가 0으로 만든 특징이 사실 중요했지만 효과 작아 떨어진 경우, 어떻게 구별?

- 구분할 수 없는 것이 정답에 가까움 — $w_j = 0$ 은 “무관하다”는 증명이 아니라 그 λ 에서 최적화가 내린 **선택의 결과**.
- 진짜 계수 3이면 $\lambda = 0.13$ 넘겨도 살아남지만 0.2였으면 같은 λ 에서 0 — 신뢰도는 “진짜 계수 $>$ 임계 λ ” 전제에 달려 있고, 그 전제는 데이터 이전에 보장되지 않음.
- 줄이는 법: 특징 **스케일링** + λ 를 6.2의 CV로 “너무 크게” 잡지 않게.

자주 묻는 질문 (2)

Q. λ 가 너무 크면?

- $\lambda \rightarrow \infty$ 이면 정규화 항이 원래 손실을 압도 $\rightarrow$ 모든 가중치(bias 제외)가 0에 수렴, “항상 평균값만 예측하는” 가장 단순한 모델로 수렴(분산 $\rightarrow 0$, 편향 극도로 큼 = 과소적합 극단).

Q. $|w|$ 가 미분불가능하면 Lasso는 최적화할 수 없는 거 아닌가요?

- 아님 — 0에서의 서브그래디언트($[-1, 1]$)로 최적조건을 정의하면 해가 존재하고 **좌표하강**이 효율적으로 찾음.
- L2는 모든 점에서 미분가능 $\rightarrow$ 구배하강·달힌형 모두 가능. Lasso는 0에서 뽀족 $\rightarrow$ 전용 알고리즘(좌표하강·내적점). 실전에서 Lasso를 그냥 `.fit()`하는 것은 이 전용 알고리즘을 쓰는 것.

Q. 정규화 계수 $\lambda/2$ vs λ , sklearn alpha?

- 기호가 서로 다름($\frac{\lambda}{2m} \|w\|^2$, $\frac{\lambda}{2} \|w\|^2$, $\lambda \sum w_j^2$). sklearn Ridge(alpha=a)는 $\frac{a}{2} \sum w_j^2$.
- **상대적 강도만** 의미 있고 절대값은 무의미 $\rightarrow$ 다른 라이브러리의 λ 를 숫자로 비교하지 말 것. “적절한 강도”는 6.2의 CV로 **데이터가 정한다**.

확인 문제 6.1

- ① 참인 것을 모두 고르라: (a) Ridge에서 $\lambda \rightarrow 0$ 이면 해는 OLS로 수렴. (b) Lasso에서 $\lambda \rightarrow \infty$ 이면 bias 제외 모든 계수 정확히 0. (c) Lasso는 항상 y 와 상관 큰 특징만 남김. (d) L1/L2의 “적당한” λ 는 train loss가 가장 작은 곳에서 고른다.

힌트: (c)는 등가 특징 예, (d)는 6.2 서두 “train loss만 보면 항상 $\lambda = 0$ ”

- ② **soft-thresholding 손계산.** $f(w) = \frac{1}{2}(2 - w)^2 + \lambda|w|$ 에서 $\lambda = 0.5$ 와 3일 때 최적 해를 $\mathcal{T}_\lambda(z)(z = 2)$ 로 구하고, “왜 0이 되고/안 되는지” $|z| \leq \lambda$ 조건으로 설명.
- ③ **서브그라디언트.** $|w|$ 의 $w_0 > 0, w_0 < 0, w_0 = 0$ 에서 서브그라디언트($1, -1, [-1, 1]$)를 구하고, $f(w) = \frac{1}{2}(z - w)^2 + \lambda|w|$ 에 대해 $w^* = 0 \iff |z| \leq \lambda$ 를 증명한 뒤 $|z| > \lambda$ 일 때 $w^* = \text{sign}(z)(|z| - \lambda)$ 확인(soft-thresholding의 증명).
- ④ **개념.** 등가 특징 예에서 Lasso가 x_1 대신 x_{2c} 를 남긴 이유를 L1 페널티 관점에서 설명(힌트: 같은 예측 만드는 (w_1, w_{2c}) 중 어느 쪽이 $|w_1| + |w_{2c}|$ 가 작나).

6.1 정리

- 정규화는 손실함수에 “가중치 크기” 페널티 한 항을 더해 유효 유연성을 낮춤 — 4.1의 언어로 λ 를 올리면 **분산을 편향과 바꾸는 것**. 살짝만 올려도 Var가 4자리수 급감할 만큼 “분산 제거”에 효율적.
- L2는 모든 가중치를 **매끄럽게** 축소, L1은 soft-thresholding($|z_j| \leq \lambda$ 이면 0)으로 약한 특징을 **정확히 0**에 스냅시켜 특징 선택을 무상으로 — 기하학적으로 “원 위” vs “마름모 꼭짓점 위”.
- bias는 관례적으로 정규화 대상에서 **제외**.

“ λ 자체는 얼마나 적당한가” — U자형 곡선의 바닥을 정직하게 고르는 절차는 6.2(교차검증)·6.3(train/val/test)에서.

6.2 교차검증: λ 를 데이터로 정한다

왜 검증 데이터가 필요한가

λ 는 사람이 직접 정해야 하는 **하이퍼파라미터** — 학습으로 정해지는 w 와 달리, 학습 시작 전에 값을 정해줘야 하는 파라미터.

- 학습 데이터 손실만 보고 고르면 **항상** $\lambda = 0$ (정규화 없음)이 제일 낮은 손실을 냄 → 의미 없음.
- **검증 데이터**(validation set)로 성능을 재야 한다.

데이터가 넉넉하지 않을 때: k-겹 교차검증

k-겹 교차검증(k-fold cross-validation): 데이터를 k 개로 쪼갬 뒤, 매번 한 조각을 검증용으로 남기고 나머지로 학습하는 것을 k 번 반복해 성능을 평균냄 — 전체를 한 번씩은 검증에도 학습에도 써보는 셈.

k-fold 분할 코드

```
def k_fold_split(data, k, fold_idx):  
    n = len(data)  
    fold_size = n // k  
    start = fold_idx * fold_size  
    end = start + fold_size if fold_idx < k - 1 else n  
    val = data[start:end]  
    train = data[:start] + data[end:]  
    return train, val
```

- fold_idx번째 블록을 검증용(val), 나머지를 학습용(train) 반환. 마지막 fold는 남은 것을 전부 포함.
- **주의: 주어진 순서 그대로 앞에서부터 자름 — 정렬된 데이터는 S 자주 하는 실수 참조.**

실전 KFold(shuffle=True) 또는 클래스 비율을 유지하는 StratifiedKFold 권장.

손으로 한 번: “아무것도 못 배우는” 모델 5-fold CV

검증 성능이 어떤 숫자가 “합리적인지” 감각을 위해, 가장 단순한 모델 — 학습 데이터의 평균만 되된다(평균모델) — 로 5-fold CV를 손계산. $y = 1, 2, \dots, 10$ 을 순서대로 2개씩 5등분:

fold	val	train 평균	fold별 MSE
0	1, 2	6.5	25.25
1	3, 4	6.0	6.50
2	5, 6	5.5	0.25
3	7, 8	5.0	6.50
4	9, 10	4.5	25.25
		평균	12.75

fold 0 풀어쓰기: $\text{train} = \{3, \dots, 10\}$ 의 평균 $= (3 + \dots + 10)/8 = 6.5$,
 $\text{MSE} = [(1 - 6.5)^2 + (2 - 6.5)^2]/2 = [30.25 + 20.25]/2 = 25.25$.

“정직한” 기준선: 진짜 평균 5.5로 예측했을 때의 $\text{MSE} = \frac{82.5}{10} = 8.25$.

CV 추정값은 “진짜 성능”이 아니다

CV 추정값(12.75)이 정직한 값(8.25)보다 53%나 높다

각 fold의 “train 평균”(6.5, 6.0, 5.5, 5.0, 4.5)이 진짜 평균 5.5 주위를 흔들리기 때문. train 평균이 1.0만큼 어긋난 fold 0에서 그 어긋남이 val 오차에 그대로 더쳐진다.

CV 추정값은 “진짜 성능”이 아니라 “검증 절차의 시뮬레이션” — 절차 안의 작은 흔들림도 포함되므로 특정 모델 하나엔 오히려 높게(비관적으로) 나올 수도.

- “CV 수치가 항상 낙관적”이라는 직관을 **바로잡자** — **CV의 낙관적 편향은 “한 모델을 평가”할 때 생기는 게 아니라, “여러 후보 중 제일 좋은 모델을 고르는” 다음 단계에서 생긴다**(§ 최적화 편향).
- 모델을 단순히 평가하는 단계(여기처럼 유연하지 않으면) — CV가 진짜 오차보다 낮게 나올 리가 없고, 실제로 높게 나올 수도.

k-fold CV로 최적 λ 찾기

```
def cross_validate_lambda(X, y, lambdas, k=5, alpha=0.01, epochs=500):
    data = list(zip(X, y))
    results = {}
    for lam in lambdas:
        val_errors = []
        for fold in range(k):
            train, val = k_fold_split(data, k, fold)
            Xtr, ytr = [d[0] for d in train], [d[1] for d in train]
            Xva, yva = [d[0] for d in val], [d[1] for d in val]
            w = ridge_gradient_descent(Xtr, ytr, lam, alpha, epochs)
            val_errors.append(mse(Xva, yva, w))
        results[lam] = sum(val_errors) / len(val_errors)
    return results
```

“학습 손실이 아니라 검증 손실로 λ 를 고른다”는 원칙을 코드로 확인.

U자형이 실제로 그려진다

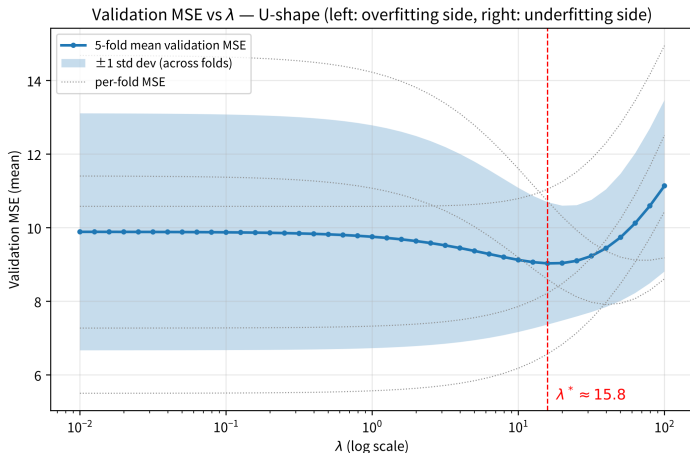
특징 15개 중 y 와 관련 있는 건 2개뿐, 나머지는 순수 잡음. 실행 예:

λ	평균 검증 MSE
0.0	12.520
0.5	10.872 ← 최적
2.0	11.780
10.0	13.740

- 정규화 전혀 없음($\lambda = 0$)도, 너무 강함($\lambda = 10$)도 검증 MSE가 오히려 나빠지고, 그 사이 어딘가에 최적점.
- 이 U자 모양 = 6.1의 편향-분산 트레이드오프가 실제 숫자로 나타난 것.

같은 종류의 데이터(15개 특징 중 2개 진짜, $n = 100$, 잡음 $\sigma = 3$)에 로그 스케일 41개 후보를 밀게 돌려 그리면 모양이 선명. Ridge는 정규방정식으로 닫혀 있어 완전한 해로만 봄.

U자형 곡선: 좌측=과적합, 우측=과소적합



λ (로그 스케일)에 따른 5-fold 평균 검증 MSE — U자형. 최소점 $\lambda^* \approx 15.8$ (MSE ≈ 9.03). 회색 점선 = fold별 MSE, 막대 = ± 1 표준편차 밴드.

1SE 규칙: 최소점은 뾰족한 꼭짓점이 아니다

- CV 곡선의 “최소점”은 추정치 — 데이터가 조금만 바뀌어도 위치가 동네를 바꿈.
- 같은 생성 과정을 시드 10개로 반복하면 각 시드가 고른 λ^* 가 **5.0~20.0까지**(약 4배) 흩어짐
→ 평탄한 대지 (plateau).

(1-standard-error rule, Tibshirani 1996)

CV MSE가 (최소점 + **fold 표준편차 1개**) 이내인 후보 중 **가장 큰 λ** — 즉 **가장 단순한 모델** — 을 고른다.

논리: “CV 평균의 불확실성은 ± 1 표준편차 급이고, 그 범위 안에서는 더 단순한 쪽을 고르는 것이 합리적 tie-breaker”.

1SE 규칙 적용: 5배 큰 λ

위 곡선에서 계산:

- 최소점 $\lambda \approx 15.8$ (MSE 9.03), fold 표준편차 1개(1.66)를 더한 허용 상한 = **10.69**.
- 이 범위 안에 드는 후보 중 가장 큰 $\lambda = \lambda \approx 79.4$ (MSE 10.59) — 최소점보다 **5배** 더 큰 λ .

실전 의미

더 작고 단순한 모델이 “통계적으로 구별되지 않는” 수준으로 좋다. 6.1의 Lasso로 치면, 정확도를 사실상 그대로 유지하면서 더 많은 특징을 0으로 만들어 **더 희소하고 해석하기 쉬운** 모델을 얻음.

손으로 한 번: 4-fold 분할 직접 나눠보기

샘플 12개(인덱스 0~11)를 $k = 4$ 로 나누면 $\text{fold_size} = 12 // 4 = 3$. 각 fold_idx 에 대해 $\text{start} = \text{fold_idx} \times 3$, $\text{end} = \text{start} + 3$ (마지막 fold만 n 까지):

fold_idx	검증용(val)	학습용(train)
0	0, 1, 2	3, 4, ..., 11
1	3, 4, 5	0, 1, 2, 6, 7, ..., 11
2	6, 7, 8	0, 1, ..., 5, 9, 10, 11
3	9, 10, 11	0, 1, ..., 8

- 4번을 전부 돌리면 인덱스 0~11 **각각이 정확히 한 번씩** 검증용으로, 나머지 세 번은 학습용으로 — “전체를 한 번씩은 검증에도 써본다”는 설명이 실제로 이런 인덱스 분할로 구현됨.
- `cross_validate_lambda`는 이 과정을 각 λ 후보마다 반복, k 번의 검증 오차를 평균 내 그 λ 를 평가.

자주 하는 실수: 섞지 않은 데이터를 fold로

`k_fold_split`은 **순서 그대로** 앞에서부터 자른다 — 데이터가 정렬되어 있으면 심각한 문제.

- 샘플 12개 중 앞 6개 = 클래스 0, 뒤 6개 = 클래스 1로 정렬되어 있으면: `fold_idx=0`의 검증셋(0,1,2)은 **클래스 0만**, `fold_idx=3`(9,10,11)은 **클래스 1만**.
- 학습에서의 클래스 비율과 검증 fold의 비율이 달라져, 검증 성능이 실제보다 훨씬 나쁘게(또는 우연히 좋게) 나옴.

분류 30개(앞 15개 클래스 0, 뒤 15개 클래스 1로 정렬), 특징 1개, $\lambda = 0$ 으로 5-fold:

분할	fold별 정확도	평균	fold 간 σ
섞지 않음(순서)	1.00, 1.00, 0.50, 0.00, 0.00	0.50	0.447
섞음(시드 42)	0.50, 0.67, 0.33, 0.33, 0.67	0.50	0.149

첫 행: 검증셋이 **어느 쪽 끝에 놓였는지가** fold별 정확도를 정해버림(1.00→0.00). 비율이 기울면 평균 자체가 왜곡.

“5-fold 평균 0.50”만 보고 비교하는 순간 — fold별 흠어짐을 볼 줄 알아야 “평균”을 읽을 수 있다.

λ 를 고르는 것 자체가 “선택”: 최적화 편향

지금까지 CV 곡선의 “최소점”을 그 λ 모델의 “진짜 성능”인 것처럼 다루었다. 절차 전체를 한 발 물러서면:

- ① 각 λ 후보마다 CV 평균 계산(데이터 사용).
- ② 그중 **가장 작은 값을 가진 λ 를 고른다** — 이것도 같은 데이터로 한 **선택**.
- ③ 그 최소점의 값을 “선택된 모델의 기대 성능”으로 보고.

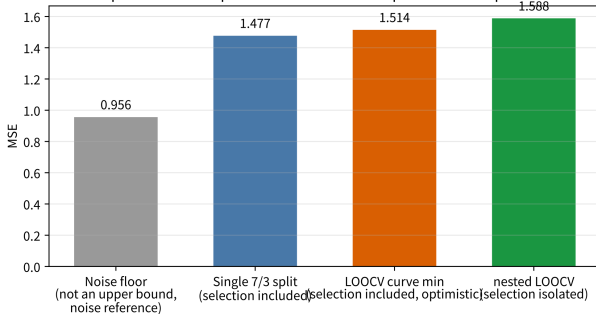
2단계는 6.3의 선택 편향(“후보 m 개 중 최고 점”을 고르면 $\mathbb{E}[\max]$ 만큼 부풀음)과 **정확히 같은 구조** — 고르는 대상이 모델이 아니라 λ 후보일 뿐.

후보가 많을수록, 각 후보의 CV 평균 간 흔들림이 클수록, **곡선의 최소점은 평균적으로 낙관적**. CV 곡선 자체가 그 데이터로 “조정된(tuned)” 결과이므로 그 최솟값은 보아온 숫자 중 제일 좋은 숫자.

이 편향을 정량화한 대표 논문: Varma · Simon (2006), “Bias in error estimation when using cross-validation for model selection”.

nested 교차검증 구조

estimates of 'final performance': optimistic bias when the optimization step touches the evaluation data



외층 fold: 검증용 점 1개(녹색). 내층: 남은 데이터로 k-fold를 돌려 λ 선택(주황 박스, “선택은 여기에서만”). 정보 흐름이 외층 검증 데이터에 닿지 않도록 내층 → 외층 방향으로만.

- 원칙적인 고치: 데이터를 **두 층으로 분리**.
- 외층 검증 데이터는 “최종 측정”에만, λ 를 고르는 모든 선택 단계는 내층에서만.

편향이 실제로 얼마인지: 숫자로

($m = 10$, 특징 1개, $\lambda \in \{0, 1, 10\}$)

추정 방법	추정 MSE	선택이 평가에 닿는가
노이즈 플로어(참고선)	0.956	—
(a) 단일 7/3 분할	1.477	평가 3개에는 안 닿음
(b) LOOCV 곡선의 min	1.514	당함 — 곡선 전체가 10개로 그림
(c) nested LOOCV	1.588	안 닿음(선택은 전부 내층 9개)

핵심은 (b) vs (c): (c)가 (b)보다 0.07 높게 — 이 갭이 “곡선 최소점을 최종 성능으로 보고한” 것에 배어 있던 낙관적 편향.

- (c)의 내부 선택: 10번 중 9번 $\lambda = 1.0$, 1번 $\lambda = 0.0$ — 선택이 데이터에 따라서 흔들리는 것 자체가 § 1SE 규칙의 경고.
- $m = 10$ · 후보 3개라 갭이 작지만 구조는 6.3과 동일 — 후보 100개, $\sigma = 1$ 이면 $\mathbb{E}[\max] \approx 2.5$ 까지 부풀어 **후보 수 · 흔들림이 커질수록 갭도 함께 커진다.**

실전 가이드라인: “항상 nested CV”가 아니다

nested CV는 보통 CV보다 k 배 가까운 학습 횟수를 먹는다.

- 보통 CV로 λ 를 **고르는 것**(selection)은 당연 — 그것이 val의 정당한 용도.
- **보고(reporting)**할 때는, 어떤 선택에도 쓰인 적 없는 **홀드아웃 데이터**(6.3의 test set)로 재는 것이 원칙.
- 홀드아웃이 없는 상황(경진대회, 데이터 전부 소진)이라면 nested CV가 “같은 데이터로 정직한 추정치를 얻는” 방법.

이 절의 λ 선택이 만드는 최적화 편향

“어떤 데이터로 무엇을 고르고, 어떤 데이터로 최종 측정하는가”를 엄격한 원리로 정리하는 것이 바로 6.3의 주제.

자주 묻는 질문 (1)

Q. k 는 몇으로?

- $k = 5$ 또는 10이 실전 기본값.
- k 가 클수록($k = m$, **LOOCV**) 각 fold 학습 데이터가 거의 전체 $\rightarrow$ 편향 줄지만, fold를 m 번 다시 학습 $\rightarrow$ 비용 $\uparrow$ · fold 간 분산 $\uparrow$.
- k 가 작을수록($k = 3$)은 빠르지만 fold 학습 데이터 작아 편향 $\uparrow$.
- 5 ~ 10이 실용적 절충점.

Q. 교차검증은 항상 해야 하나요?

- 데이터가 아주 많으면(수백만 개) 한 번만 train/val로 나뉘도 검증셋이 충분히 커 안정적인 평가.
- 적을수록(수백~수천) 한 번의 분할이 우연히 “쉬운/어려운” 검증셋을 뽑을 위험이 커 $\rightarrow$ 교차검증의 가치 $\uparrow$.

자주 묻는 질문 (2)

Q. 경진대회(Kaggle)에서 test 라벨이 안 공개되면?

- 공개 **public leaderboard**는 사실상 val, 비공개 **private**는 test 역할.
- public 점수로 λ 를 골라 계속 제출하면, public을 “20번 봤다”는 뜻 $\rightarrow$ $\S$ 최적화 편향
관점에서 public 점수는 **부풀려진** 추정치. private의 제출 횟수 제한이 바로 이 선택 편향을
통제하기 위함.
- **교훈: 같은 검증 조각으로 비교한 후보가 m 개면, 그 검증 점수는 m 개 중 max를 본 셈.**

Q. k-fold CV 평균은 “진짜 성능”의 공정한(무편향) 추정치?

- “한 모델 평가” 용도라면 같은 분포의 새 데이터 기대 오차에 대한 합리적 추정치.
- “여러 λ 후보 중 최솟값을 고르는” 용도로 쓰면 그 최솟값은 평균적으로 낮게(낙관적) —
정직한 숫자가 필요하면 선택에 쓰지 않은 test(6.3)나 nested CV를 쓴다.

확인 문제 6.2

- 1 샘플 10개(인덱스 0~9)를 $k = 5$ 로 `k_fold_split`. `fold_idx=2`일 때 `val`에 들어가는 인덱스를 쓰고, 5개 fold를 모두 돌렸을 때 각 인덱스가 `val`에 총 몇 번 등장하는지 확인.
- 2 평균모델 예($y = 1, \dots, 10$, 순서 2개씩 5등분)에서 fold 0과 4의 fold별 MSE를 각각 계산하고 5-fold 평균을 구해, 정직한 8.25와 CV 12.75의 차이를 한 문장으로 설명.
- 3 아래 λ 5개에 대한 5-fold 평균 MSE: (가) 최소점은? (나) 최소점 $\lambda = 1.0$ 의 fold별 MSE = [4.2, 4.4, 4.6, 4.8, 5.0]일 때 1SE 규칙(최소점 평균 + fold 표준편차 1개 이내인 후보 중 **가장 큰** λ)으로 고를 λ 는?

[λ : 0.1 0.3 1.0 3.0 10.0] / [MSE : 5.00 4.80 **4.60** 4.70 5.20]

- 4 λ 후보 20개 CV의 최소점(MSE 3.12)을 “배포 모델의 기대 성능”으로 보고했다. 이 주장이 왜 낙관 편향이 있는지 § 최적화 편향의 구조로 설명하고, 정직한 추정치를 얻는 두 방법(하나는 데이터를, 하나는 계산을 더 요구)을 쓰라.

6.2 정리

이 절은 두 가지를 손에 넣었다.

- ❶ **절차** — k-fold CV로 전체를 “한 번씩은 검증에도” 쓰면서, 학습 손실이 아니라 검증 손실이 가장 작은 λ 를 고른다: U자형 곡선은 6.1의 편향-분산이 실제 숫자로 그려진 것이고, 1SE 규칙은 “최소점” 단일 수치에 대한 건강한 불신을 **제도화**한 것.
- ❷ **경고** — “여러 후보 중 최솟값을 고르는” 행위 자체가 또 하나의 선택이므로, CV 곡선의 최소점은 곧바로 “최종 성능”이 될 수 없다.

낙관적 편향을 격리하는 방법 — 선택에 쓰지 않은 test, 또는 nested CV — 은 6.3의 train/val/test 분리 원리가 이 절과 맞닿는 지점.

6.3 train/val/test 분리 원칙 실전

왜 3분할이 필요한가: 배우다와 재다를 나누어야

머신러닝 파이프라인은 두 가지 완전히 다른 일을 동시에 한다:

배우는 일 train 데이터로 모델의 파라미터를 정하는 것.

재는 일 그 모델이 보지 못한 데이터에서 얼마나 잘하는지 평가.

이 둘을 같은 데이터로 하면

모델이 “재는 데이터”까지 보고 만들어졌으므로, 그 점수는 “학습을 얼마나 잘했는가”가 아니라 “기억을 얼마나 잘했는가”를 재게 된다. 4.1의 과적합(“train에서 완벽, test에서 참사”)과 정확히 같은 구조.

6.1의 정규화(λ)와 6.2의 교차검증은 이 문제를 완화했지만 **역할 분리** 자체를 정한 곳은 아직 없다.

역할 분리: 각 조각이 딱 한 역할만

데이터를 세 조각으로 나눠, 각 조각이 서로 다른 질문에 답하도록 **각각 딱 한 역할만** 쓰기로 약속:

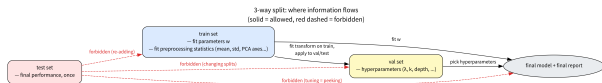
- **train**은 **배운다** — 학습 데이터로 파라미터(w)를 정.
- **val**은 **모델을 고른다** — 하이퍼파라미터 (λ , kNN의 k , 트리 깊이 등)를 정.
- **test**는 **딱 한 번만** 최종 점수를 준다.

한 줄 압축

“정보의 흐름 방향을 정한다”. train은 val·test로, val은 최종 모델로 정보가 흐를 수 있지만, **test에서는 어디로도 정보가 흘러나와서는 안 된다**. test는 파이프라인의 마지막 관문 — 한 번 지나가면 돌아올 수 없다.

테스트를 하이퍼파라미터 튜닝에 쓰면 “부정행위” (test를 몰래 봐서 고른 모델)와 다를 게 없다.

정보 흐름 다이어그램



허용된 흐름(실선)은 train에서 시작해 val을 거쳐 최종 모델로 흐르고, 테스트(빨간 점선)에서 나가는 모든 화살표는 금지.

- **train** → **val / test**: 전처리 통계량(평균·표준편차, PCA 축)을 train으로만 fit하고 val·test에는 apply — 허용되는 유일한 교차.
- **train** → **모델**: w 를 fit. **val** → **모델**: 하이퍼파라미터를 고르는 방향.
- **test** → (**val / train / 모델**): 모두 금지 — 전부 “테스트를 살짝 훑쳐보았다”는 뜻.

이 다이어그램이 이 절의 지도. 다음 세 절(선택 편향, 시계열, 그룹 분할)은 이 지도의 빨간 점선을 밟아봤을 때 실제 숫자가 어떻게 변하는지를 보여준다.

3분할이 실제로 깨지는 흔한 실수 세 가지

- ① **테스트로 여러 모델을 비교**해 가장 잘 나온 걸 고르는 것 (“살짝 훑쳐보기”).
- ② **검증 데이터 없이** 테스트 데이터로 하이퍼파라미터까지 같이 튜닝(검증과 테스트를 사실상 섞어버림).
- ③ 데이터를 나누기 **전에** 정규화(스케일링) 같은 전처리 통계량(평균, 표준편차)을 **전체 데이터**로 계산 — test 정보가 전처리 단계에서 이미 학습에 스며들(**data leakage**).

세 실수가 모두 같은 범주 — “**test 정보가 다른 곳으로 흘러나감**” — 에 속한다. 겉모양은 달라(모델 고르기 / 전처리 순서 / ...) 보여도 **다 같은 정보 흐름의 역주행**.

더 좋은 질문

“이 코드에 leakage가 있는가?”보다: “**test(또는 val)의 정보가 학습이나 선택 단계에 닿았는가?**”

아래 § 에서 이 질문을 **숫자로** 답할 수 있는 도구를 하나씩 만든다.

Ch. 8, 16 팀 프로젝트와 ML2에서도 이 3분할 원칙을 그대로 지켜야 한다.

손으로 한 번: 누수를 숫자로

세 번째 실수(정규화 통계량을 전체 데이터로 미리 계산)가 어떤 왜곡을 만드는지. 값 1 ~ 10을 앞

	기준	평균	표준편차
7개(train) · 뒤 3개(test)로 나눔:	전체 10개(누수)	5.5	2.872
	train 7개만(올바름)	4.0	2.000

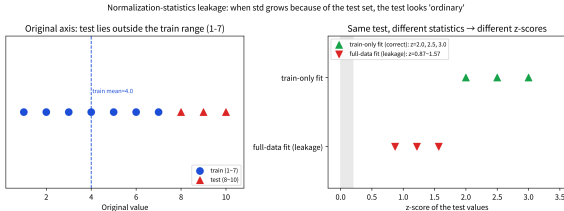
test = [8, 9, 10]를 각각의 통계량으로 표준화:

전체(누수) : [0.870, 1.219, 1.567] train(올바름) : [2.0, 2.5, 3.0]

- 전체 데이터로 미리 계산한 평균 · std를 쓰면 test 값이 실제보다 훨씬 “평범(z값 작음)”해 보인다 — std 자체가 test의 큰 값들 때문에 이미 커져 있기 때문(2.872 vs 2.0).

배포 시나리오: 미래를 볼 수 없다

- 모델을 배포하는 시점에는 **미래에 들어올 데이터(test에 해당)**를 아직 볼 수 없음 → 애초에 그 통계량을 계산에 쓸 수 없다.
- **train 통계량만으로 표준화**가 실제 배포 상황을 정확히 흉내 낸 것이고, 전체 데이터로 계산하는 것은 “미래를 몰래 들여다본” 값.
- 이 차이가 검증 성능을 **실제보다 낙관적으로 부풀리는** 방향으로 작용하는 경우가 많다.



좌측: 원본 축에서 test(8~10)는 train 범위(1~7)를 벗어남. 우측: 전체로 fit한 std(2.87)로 나누면 test의 z가 0.87~1.57로 “평범”, train만 fit한 std(2.0)로 나누면 2.0~3.0으로 “극단”.

왜 std가 test 때문에 커지는가

scikit-learn의 표준 구현

```
# 올바른 순서 : 으로는 train fit, val/test 만 transform
scaler = StandardScaler().fit(X_train)
X_val = scaler.transform(X_val)
X_test = scaler.transform(X_test)
```

.fit_transform(X_all)을 val·test에 쓰면 “누수” 결과와 동일한 위장이 일어난다.

“살짝 훑쳐보기”의 통계: 선택 편향

“테스트로 여러 후보를 돌려보고 제일 잘 나온 걸 고른다” — 이 한 줄이 정량적으로 얼마나 낙관적인지 확률로 계산.

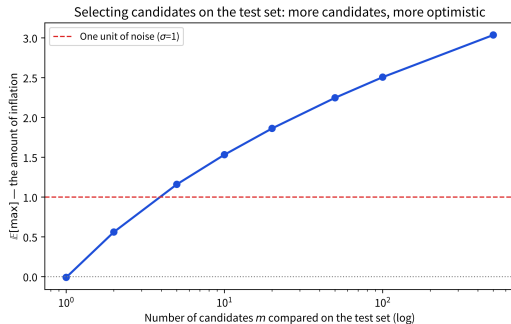
- 각 후보의 테스트 성능을 (진짜 성능 + 우연한 잡음)으로 모델링, 잡음을 독립 정규분포 $\mathcal{N}(0, \sigma^2)$.
- 후보 m 개 중 **최고 점**을 고르는 행위 = m 번의 잡음 뽑기 중 max를 고르는 것과 같음.
- m 개 독립 정규분포의 max는 평균(=0, 진짜 성능)보다 **항상 위쪽으로 치우침** — m 이 커질수록 max는 더 위로.

이 편향을 정량화한 대표 논문: Varma·Simon (2006).

$\mathbb{E}[\max]$ 를 숫자로

$\sigma = 1$, 진짜 성능 0인 경우 4만 번 시뮬레이션. 후보 수 m 에 따른 “고른 최고 점”의 평균:

m	1	2	5	10	20	50	100	500
$\mathbb{E}[\max]$	≈ 0	0.56	1.16	1.53	1.86	2.25	2.51	3.04



후보 수 m (로그 축)에 따른 $\mathbb{E}[\max]$ — 잡음 한 개($\sigma = 1$)를 붉은 점선으로. $m = 50$ 이면 잡음의 2배 이상, $m = 100$ 이면 2.5배 부풀어 오름.

val에 걸리는 편향은 정직하다

- 이 편향이 **test가 아니라 val에 걸리는 한** 정직 — val은 “모델을 고르는 역할”이라 선택 편향을 예상대로 먹고.
- **test는 한 번만 마지막에** 쓰이므로 그 편향에 노출되지 않음.
- 그런데 test로 고르면 재야 할 대상(test)이 고르는 과정에 쓰여서 test 자체가 $\mathbb{E}[\max]$ 를 그대로 먹음 — **평가자가 동시에 선수 선발위원이 되는 셈.**

σ 는 “성능 측정의 흔들림” 크기 — 실제에선 σ 를 곱하면 됨(잡음 3배면 부풀림도 3배).

한 줄 교훈: 후보가 m 개면 “제일 좋은 test 점”은 진짜 성능보다 $\mathbb{E}[\max]$ 만큼 평균적으로 높다. m 이 1이어야 정직하고, 크면 그 수치는 하한이 아니라 낙관적 상한에 가까움.

자주 하는 실수: 시계열을 무작위로 나누기

환자에게 있는 “단위” 문제의 다른 얼굴이 **시간 순서**. 시계열은 데이터가 이미 시간순 정렬되어 있을 때 무작위 shuffle이 구조를 깨는 방향으로 작용.

추세(매일 +1)가 있는 일일 데이터 365일, kNN:	분할 방식	test RMSE
	무작위 분할	0.81
	시간순 분할(뒤 20% = test)	44.1

- **무작위**: test 점들이 시간축 곳곳에 흩어져 각 test 점 근처에 train 점이 항상 있음(과거든 미래든) → kNN이 잘 맞음. 그런데 **미래 값을 과거 값으로 “예측”**한 것 — 실전(미래는 아직 없음)에선 성립하지 않음.
- **시간순**: test가 train의 시간 범위(horizon)를 벗어나 kNN이 밖으로 외삽, 추세를 못 쫓아 RMSE 급등.

교훈

이 **44.1**이 실제 일반화 성능에 가까운 값. 시계열은 무작위가 아니라 **시간순으로** 나눠야 하고, 더 근본적으로 **“실전에서 겹치지 않을 것”을 같은 조각에 두지 말라**. `train_test_split(shuffle=False)` / `TimeSeriesSplit`. 그룹(환자·기기·지점): `GroupKFold`.

실습: diabetes 3분할 파이프라인 전체

```
X, y = load_diabetes().data, load_diabetes().target # 건442, 개10 특징
# 1) 단계2 분할3: 를40% (val+test)로, 그를 40% 절반씩
Xtr, Xtemp, ytr, ytemp = train_test_split(X, y, test_size=0.4, random_state=0)
Xva, Xte, yva, yte = train_test_split(Xtemp, ytemp, test_size=0.5, random_state=0)
# 2) val 로만RMSE lambda 고르기선택 ( 편향은에서만 val)
lams = [0.1, 1.0, 10.0, 100.0, 1000.0]
val_rmse = {lam: np.sqrt(mean_squared_error(
    yva, Ridge(alpha=lam).fit(Xtr, ytr).predict(Xva))) for lam in lams}
best = min(val_rmse, key=val_rmse.get)
# 3) 그로 lambda 만train fit, test RMSE 한번만
final = np.sqrt(mean_squared_error(
    yte, Ridge(alpha=best).fit(Xtr, ytr).predict(Xte)))
```

(4) 대조로 (잘못) test로 λ 를 고르면 어떤 차이가 나는지 본다.

실행 결과: 핵심은 숫자보다 절차

크기: train=265 val=88 test=89

val RMSE per lambda: {0.1: 58.07, 1.0: 61.75, 10.0: 73.04,
100.0: 77.49, 1000.0: 78.07}

val로 고른 lambda = 0.1 -> 최종 test RMSE (한 번만) = 53.79
(잘못) test로 고르면 lambda = 1.0

핵심

val로 고른 λ 와 test로 고른 λ 는 (이 작은 데이터에서) 같을 수도, 다를 수도 있다 — 위 실행에선 0.1 vs 1.0으로 다름.

우연히 같아도 원칙적으로 test를 선택에 썼으므로 그 test 수치는 선택편향의 $\mathbb{E}[\max]$ 를 먹고 있는 **부풀려진 추정치**.

같은 모델·같은 코드인데도 “ λ 를 어디서 고르느냐”만 달라져 최종 test 점의 신뢰성이 달라짐 — 3분할은 “코드가 아니라 절차”의 문제.

2단계 3분할은 정말 겹치지 않는가

실전에선 한 번에 3등분보다 **2단계**로 나눔: 전체 40(val+test)로 떼고, 그 40 샘플 100개(0~99)를 “앞 60=train, 뒤 40=temp” 1차 분할 후 temp(60~99)를 “앞 20=val, 뒤

	조각	인덱스 범위	개수
20=test” 2차:	train	0~59	60
	val	60~79	20
	test	80~99	20

- 세 범위가 **서로 겹치지 않는지**(train/val, train/test, val/test 두루) — 교집합이 빈집합인지.
- $\text{val} \cup \text{test}$ 가 “처음 떼어둔 뒤 40%”(60~99)와 **등치**하는지.

“한 번에 3등분”과 “2단계 분할”이 동일한 disjoint 조건을 만족함을 보이면, 2단계 분할을 써도 3분할 원칙을 위반하지 않음이 확인.

유명한 사례: 환자 단위 누수

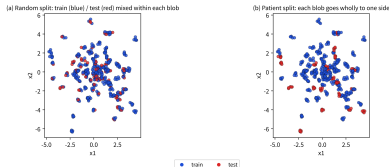
의료 영상 AI에서 실제로 자주 발견되는 누수 — **같은 환자의 여러 장 스캔 중 일부는 train, 일부는 test에 무작위로 흩어지는 경우.**

- 같은 환자의 스캔은 서로 매우 비슷(같은 신체 구조, 같은 장비).
- 모델이 test에서 “새로운 환자를 진단”하는 게 아니라 “이미 학습에서 본 그 환자를 살짝 다른 각도에서 다시 알아보는” 셈이 됨.
- 논문에 보고된 정확도가 실제 새로운 환자에게 적용했을 때보다 **부풀려짐**.

환자 100명, 각 5장 스캔, kNN:

분할 방식	test RMSE
무작위 분할(이미지 단위)	1.04
환자(그룹) 분할	3.65

Patient-level leakage: the 5 images of the same patient cluster into one blob



환자를 “외운다” — 왜 1.0이 “성취”가 될 수 없는가

- 무작위 분할 RMSE(≈ 1.0)가 환자별 값의 표준편차(2.0)보다 훨씬 낮음.
- 새로운 환자를 예측하는 것은 이론적으로 $\sigma\sqrt{2} \approx 2.8$ 아래로 내려갈 수 없는데 1.0을 “성취”했다.
- 이걸 모델이 관계를 배운 것이 아니라 환자를 외운 것.

```
from sklearn.model_selection import GroupKFold
groups = np.repeat(np.arange(100), 5) # 환자100 x 장5, 그룹키환자=
gkf = GroupKFold(n_splits=5)
for tr_idx, te_idx in gkf.split(X, y, groups=groups):
    pass # 같은환자의장은 5 항상 train 또는 test 중하나로만
```

GroupKFold: 각 fold마다 다른 환자를 배정 → 같은 환자의 5장은 절대 train/test에 걸리지 않음. 6.2의 “fold가 구조를 깨면 안 된다”는 원칙의 그룹 버전.

자주 묻는 질문 (1)

Q. 데이터가 너무 적어 3분할하면 각 조각이 너무 작아지는데?

- test를 따로 떼는 대신, 6.2의 교차검증으로 “학습+검증”을 반복하고, 정말 최종 성능이 필요할 때만 별도로 확보한 **작은 test로 딱 한 번** 확인하는 절충안.
- 극도로 적으면 LOOCV까지 고려. 그래도 **test를 완전히 없애고 검증 성능만으로 최종 성능을 주장하는 것은 피해야** 한다 — 검증 성능은 이미 하이퍼파라미터 선택에 “간접적으로” 쓰였기 때문.
- **몇 대 몇?** 정해진 정답 없음 — 60/20/20이나 70/15/15가 흔한 시작점. 아주 많으면(수백만) val·test가 각각 1~2%만 되어도 안정적.

Q. 교차검증(6.2)을 쓰면 test는 굳이 필요 없지 않나요?

- 보통은 그렇지만, “정확히 한 번”의 최종 성능이 필요한 (논문 제출, 배포 전 검증) 상황에서선 필요.
- k-fold CV는 val 역할(하이퍼파라미터 고르기), test는 최종 판정(아무 선택 없이 처음 보는 데이터로 검증). CV만 있으면 “고르는 과정”만 있고 “판정”이 없다.
- 특히 **CV를 여러 번(λ 마다, 시드마다) 돌리면서** test 점수를 보고 결정 내리면, test가 다시 선택에 쓰임.

자주 하는 실수: 전처리 fit 시점과 “살짝만 조정”

Q. 전처리(스케일링, 임베딩, 결측치 대체)를 모델과 함께 파이프라인으로 묶으면 leak이 안 생기나요?

- 그 반대 — 파이프라인으로 묶는 것이 leak을 막는 정석.
- `Pipeline([('scaler', StandardScaler()), ('model', Ridge())])`에서 `pipeline.fit(X_train, y_train)`만 하고 `pipeline.predict(X_test)`를 쓰면, scaler가 train만 fit되어 val/test에는 train 통계량이 자동 적용.
- 반대로 `scaler.fit_transform(X_all)`을 분할 전에 하면 S 손계산의 “누수” 열이 그대로 재현.
- 전처리 fit은 항상 분할 이후, train 조각에만.

Q. val로 λ 고르고 test로 “한 번” 확인했는데, test가 기대에 못 미쳐 val을 다시 보고 살짝만 조정했다. 이 test 수치 신뢰?

- 아님. “살짝만” 조정하는 순간 test는 선택 과정에 쓰였다 — 후보 수 m 이 1에서 2로 늘어난 셈이라 편향이 0에서 $\mathbb{E}[\max \text{ of } 2]$ 로 뛴음.
- “한 번만”은 정말로 한 번이라는 뜻 — 한 번 보고 마음에 안 들면 같은 test를 다시 보며 “이번엔 진짜 마지막으로”라고 해도 이미 두 번째.

확인 문제 6.3

- ① 테스트 정보가 다른 단계로 흘러나가는 것에 모두 체크: (a) val RMSE로 λ 고르기 (b) test RMSE로 λ 고르기 (c) `StandardScaler().fit(X_all)` 후 train/val/test에 transform (d) train으로 fit한 스케일러로 test transform (e) test에서 제일 잘 맞는 모델을 보고, “더 잘 맞추려고” train에 test를 추가로 넣기.
- ② 모델 후보 10개 $\times$ λ 후보 10개 = 100개 조합을 테스트로 평가해 최고 조합 고름. 잡음 $\sigma = 0.5$ 라면, “고른 조합의 test 점”은 진짜 성능보다 **평균적으로** 얼마나 높을까? (힌트: $\mathbb{E}[\max]$ 를 σ 로 곱해라.)
- ③ 아래 2단계 3분할 중 원칙 위반: (가) 80/20 $\rightarrow$ val/test 10/10, (나) 80/20 $\rightarrow$ test에서 제일 좋은 λ 골라 train/val에 다시 적용, (다) train으로 fit한 PCA로 val/test 투영.
- ④ “데이터 100개뿐이라 3분할하면 test가 20개밖에 안 돼서 val을 빼고 80/20만 쓰고, test로 λ 를 고르되 최종 수치는 그 test의 5-fold 평균으로 보고하자”는 제안을 평가 — 어떤 점이 원칙을 지키고, 어떤 점이 위반하는가?

6.3 정리

- 정규화는 “모델을 덜 유연하게 만들어 분산을 줄인다”는 4.1의 편향-분산 원리를, 손실함수에 항 하나를 더하는 것만으로 실제로 조절 가능하게 만든 도구 — 그리고 그 조절 손잡이(λ) 자체는 교차검증이라는 또 다른 절차로 정해야 한다.
- 그 교차검증/분리 절차 자체가 이 절의 주제: **train으로 w , val로 λ , test로 딱 한 번의 최종 점수.**
- 이 세 역할을 한 조각이 두 가지로 겸하면(특히 test가 재는 것과 고르는 것을 겸하면), 선택편향이 보여준 것처럼 보고된 숫자는 진짜 성능이 아니라 **잡음의 max**가 되어버린다.

3분할은 “데이터를 셋으로 쪼갬다”는 기계적 절차가 아니라, **정보의 흐름에 방벽을 세우는 일.**

이번 주차 정리

- ① **6.1 페널티의 모양이 결과를 바꾼다** — λ 를 올리면 분산을 편향과 바꿈. L2는 매끄러운 축소, L1은 soft-thresholding으로 **정확히 0**에 스냅시켜 특징 선택을 무상으로(원 vs 마름모). bias는 정규화에서 제외.
- ② **6.2 학습 손실이 아니라 검증 손실로 λ 를 고른다** — k-fold CV로 U자형 곡선을 그리며, 1SE 규칙으로 “최소점” 단일 수치에 대한 불신을 제도화. “여러 후보 중 최솟값을 고르는” 행위 자체가 선택 → **최적화 편향**, nested CV로 격리.
- ③ **6.3 train/val/test 3분할** — train은 배운다, val은 고르고, test는 **딱 한 번**. “정보 흐름의 역주행”(leakage)을 숫자로: 선택 편향($\mathbb{E}[\max]$), 전처리 통계량 누수, 시계열·환자 단위 분할.

다음 주 — Chapter 7. 트리 기반 모델

정규화·교차검증·3분할은 다음 장의 트리·앙상블(GBDT) 튜닝이 올라설 **초석**. 트리의 복잡도 손잡이는 가중치 페널티가 아니라 “트리 깊이”, “분할 횟수”가 되지만, “수많은 후보 중 어떤 설정이 최적인가?”라는 질문은 **똑같이** 남는다 — 그 답을 만드는 틀(검증, 교차검증, train/val/test)이 바로 이 장에서 지은 것이다.